

EcoSphere — Project Handoff Documentation

Project: EcoSphere — Environmental Intelligence Rover (Hazard Classification System)

Hardware: Arduino UNO Q dual-brain architecture (Qualcomm QRB2210 for AI inference, STM32U585 for real-time fail-safe control) **Purpose:** Tiered hazard classification (safe / elevated / critical) from gas sensor data, built as the centerpiece submission for the Arm AI Optimization Challenge (deadline Aug 14, 2026) **Location on disk:**

`Documents\competition_project\ai_layer\sensor_tilt`

1. What this project does

EcoSphere reads gas sensor data and classifies environmental hazard level into three tiers — **safe**, **elevated**, and **critical**. The AI/inference logic runs on the QRB2210 (Linux side of the UNO Q); real-time fail-safe control is handled separately on the STM32U585 core. The pipeline also includes a **drift demonstration** — showing how the model's predictions degrade or adapt when the input data distribution shifts over time, which is important for judging "model quality under real-world conditions."

Two model types were trained and compared:

- A **Random Forest** classifier (`hazard_model.joblib`)
 - A **Keras neural network**, converted to **TFLite** for on-device deployment (`hazard_model.tflite`)
-

2. Pipeline structure (script execution order)

The scripts are numbered — **run them in this order** to reproduce the pipeline from scratch:

Step	Script	What it does (inferred from name — confirm/expand below)
00	<code>00_load_real_gas_data.py</code>	Loads real (non-synthetic) gas sensor readings as a baseline/reference dataset
01	<code>01_generate_data.py</code>	Generates the synthetic training dataset (<code>hazard_dataset</code>)
02	<code>02_feature_engineering.py</code> (+ <code>feature_engineering_helper.py</code>)	Builds features from raw sensor data (<code>hazard_dataset_featured</code>)
03	<code>03_train_model.py</code>	Trains the baseline model (likely the Random Forest → <code>hazard_model.joblib</code>)
04	<code>04_predict_live.py</code>	Runs live/real-time prediction using the trained model
05	<code>05_drift_demonstration.py</code>	Demonstrates data drift affecting classification
06	<code>06_drift_curve.py</code>	Produces the drift curve visualization/data (<code>drift_curve_results</code>)
07	<code>07_drift_correction_comparison.py</code>	Compares model performance with/without drift correction (<code>drift_correction_comparison</code>)
08	<code>08_master_pipeline.py</code>	Master script — likely runs the full pipeline end-to-end (start here if you just want to reproduce everything)
09	<code>09_train_keras_model.py</code>	Trains the Keras neural network version of the classifier
10	<code>10_convert_to_tflite.py</code>	Converts the trained Keras model to TFLite (<code>hazard_model.tflite</code>)
11	<code>11_tflite_inference.py</code>	Runs inference using the TFLite model (this is what should run on the QRB2210 in the actual deployment)

⚠ **Note for whoever picks this up:** the "what it does" column above is inferred from filenames and prior context, not from reading the code directly. Before handing this off, it's worth adding a one-line docstring/comment at the top of each script confirming what it actually does — that will save the next person a lot of guessing.

3. Supporting files

File	Purpose
<code>hazard_dataset</code> (.xlsx)	Raw synthetic dataset
<code>hazard_dataset_featured</code> (.xlsx)	Dataset after feature engineering
<code>real_gas_sensor_data</code> (.xlsx)	Real-world sensor readings used for validation/comparison
<code>hazard_model.joblib</code>	Trained Random Forest model
<code>hazard_model.tflite</code>	Trained + converted TFLite model (for on-device deployment)
<code>label_classes.npy</code>	Class label mapping (safe/elevated/critical)
<code>norm_mean.npy</code> , <code>norm_std.npy</code>	Normalization statistics applied to input features before inference — required at inference time , don't lose these
<code>drift_curve_results</code> (.xlsx), <code>drift_correction_comparison</code> (.xlsx)	Output results from the drift analysis scripts
<code>archive/</code>	Older/superseded files — probably safe to ignore, but confirm before deleting anything
<code>__pycache__</code> /	Auto-generated Python cache — safe to ignore/delete

4. Environment setup (known issue — READ FIRST)

This project must be run in a Python 3.11 environment. TensorFlow does not yet officially support Python 3.14, which is what may be installed by default. Attempting to run the Keras/TFLite scripts (steps 09–11) in Python 3.14 will fail.

Recommended setup (using Miniconda, avoids Windows PATH issues):

```
bash

conda create -n ecosphere python=3.11 -y
conda activate ecosphere
pip install tensorflow numpy scikit-learn pandas matplotlib joblib
```

Then run scripts from within the `sensor_tilt` folder in that activated environment.

5. What's done vs. what's still open

Done:

- Full pipeline from synthetic data generation through Random Forest and Keras/TFLite model training
- Drift demonstration and drift-correction comparison
- TFLite conversion for on-device deployment

Open / in progress (as of this handoff):

- Confirming the Python 3.11 environment runs the full pipeline cleanly end-to-end after the 3.14 compatibility issue
 - INT8 quantization of the TFLite model (for the Arm challenge's "model size/speed" optimization criteria) — not yet done
 - On-device latency benchmarking on the actual QRB2210 hardware — not yet done
 - Documentation of the STM32U585 ↔ QRB2210 hand-off (dual-brain architecture) for the competition writeup — not yet done
-

6. For the person taking this over

Start by:

1. Setting up the Python 3.11 environment (Section 4)
2. Running `08_master_pipeline.py` to confirm everything still executes end-to-end
3. Reviewing this doc against the actual code and correcting anything inferred above that turns out to be wrong

If anything in Section 2's table is inaccurate, please update it directly — this doc is meant to reflect ground truth, not assumptions.